

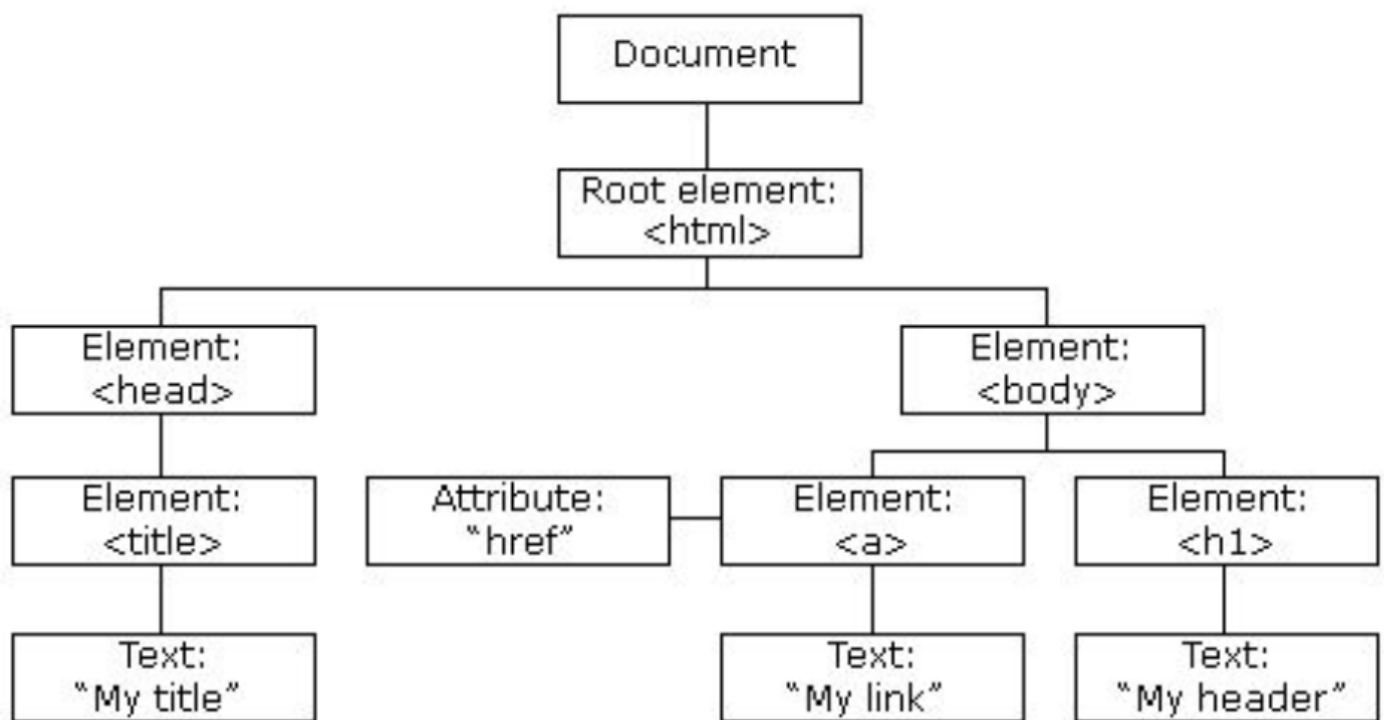
Conhecer as características da linguagem JavaScript

Aprender a utilizar JavaScript com DOM HTML

Como vimos na aula anterior, uma das funcionalidades providas pelo JavaScript é a manipulação do *Document Object Model* (DOM), ou Modelo de Documento por Objetos.

O DOM é uma interface de programação para os documentos HTML. Ele representa a página de forma que os programas possam alterar a estrutura do documento, modificando seu estilo e conteúdo.

Cada página carregada em um navegador possui um objeto chamado Document, conforme exemplificado na figura a seguir.



Dessa forma, a interface Document serve como um ponto de entrada para o conteúdo da página (Ex.: tags **<body>** e **<p>**).

Veja a seguir um resumo dos principais conceitos relacionados ao DOM HTML:

- **Árvore de elementos:** o DOM HTML organiza a estrutura de uma página web como uma árvore hierárquica, na qual o elemento **<html>** é a raiz e os elementos **<head>** e **<body>** são seus filhos. Outros elementos HTML, como **<div>**, **<p>**, **<a>**, etc., são aninhados dentro desses elementos pai, formando uma estrutura em árvore que representa a página.

- **Elementos HTML como objetos:** no DOM HTML, cada elemento HTML é representado como um objeto em JavaScript. Assim como é apresentado em disciplinas como Programação Orientada a Objetos (POO), tais objetos possuem propriedades e métodos que podem ser usados para acessar e manipular o conteúdo e os atributos dos elementos.
- **Acesso aos elementos:** possibilita acessar elementos HTML no DOM HTML usando seletores, como `getElementById`, `getElementsByClassName`, `getElementsByTagName`, etc. Esses métodos permitem que você selecione elementos específicos na estrutura DOM e faça a alteração do código HTML via JavaScript.
- **Manipulação do conteúdo:** possibilita alterar o conteúdo de elementos HTML, como texto e atributos, usando propriedades como `innerHTML`, `value`, `setAttribute`, `getAttribute`, entre outras.
- **Eventos:** o DOM HTML permite a vinculação de eventos a elementos HTML. Permite
 - usar o método `addEventListener` para ouvir eventos, como cliques de mouse, envios de formulários e digitação, e responder a esses eventos com funções de retorno.
- **Navegação na árvore:** Você pode percorrer a árvore DOM para acessar elementos pai, filhos e irmãos. Isso é útil para navegar pela estrutura da página e acessar elementos relacionados.

A seguir, veremos os seguintes exemplos de APIs em scripting de páginas Web usando a API DOM:

- `document.getElementsByTagName(name)`
- `document.getElementById(id)`
- `element.getAttribute()`
- `document.createElement(name)`
- `parentNode.appendChild(node)`
- `document.createTextNode(data)`
- `element.innerHTML`
- `element.addEventListener()`

document.getElementsByTagName(name)

Refere-se a parte da API DOM em JavaScript que permite selecionar elementos HTML em uma página da web com **base em seus nomes de tag** (e.g., p, h1, div). Essa função retorna uma coleção de todos os elementos com a tag especificada.

PRATICANDO:

- Para verificar o uso da API DOM, crie um arquivo chamado index.html, contendo a estrutura a seguir.

```
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Exemplos - JavaScript</title>
  </head>
  <body>
    <main>
      <section id="inicio">
        
      </section>
      <section>
        <h1>Objetivos de Desenvolvimento Sustentável</h1>
        <p id="desc">
          Em 2015, a ONU propôs aos seus países membros uma nova agenda de
desenvolvimento
          sustentável para os próximos 15 anos, a Agenda 2030, composta pelos 17
Objetivos de
          Desenvolvimento Sustentável (ODS).
        </p>
        <h2>Conheça mais</h2>
        <p>
          Os Objetivos de Desenvolvimento Sustentável são um apelo global à ação para
acabar com a
```

```

        pobreza, proteger o meio ambiente e o clima e garantir que as pessoas, em
    todos os
        lugares, possam desfrutar de paz e de prosperidade.
    </p>
</section>
<section>
    <button onclick="mudarCor()">Alterar cor</button>
</section>
</main>
<script type="text/javascript">
    //insira o código aqui
</script>
</body>
</html>

```

Por exemplo, se você deseja selecionar todos os elementos `<p>` (parágrafos) em uma página da Web, você pode inserir o seguinte código dentro da tag **<script>** em seu arquivo `index.html`:

```

const textos = document.getElementsByTagName("p");
console.log(textos);

```

Esse comando percorrerá o DOM e retornará uma coleção de todos os elementos `<p>` na página, atribuindo-os para a variável de nome `textos`.

Ao visualizar o código via Live Server e acessar o console (via `ctrl + shift + j`), você verá que foi retornado via `console.log` os elementos `p` encontrados, conforme a imagem a seguir.

```

> const textos = document.getElementsByTagName("p");
  console.log(textos);
  ▼ HTMLCollection(2) [p#desc, p, desc: p#desc] ⓘ
    ▶ 0: p#desc
    ▶ 1: p
    ▶ desc: p#desc
      length: 2
    ▶ [[Prototype]]: HTMLCollection
  < undefined

```

elementos `<p>`
que estão no HTML

Como o conteúdo da página continha dois parágrafos, foram apresentados dois elementos do tipo `<p>` para o console. Você pode utilizar a seta de cada elemento

para identificar as propriedades e os valores pertencentes a eles, conforme o exemplo a seguir.

```
▼ HTMLCollection(2) [p#desc, p, desc: p#desc] ⓘ  
  ▼ 0: p#desc  
    accessKey: ""  
    activeViewTransition: null  
    align: ""  
    ariaActiveDescendantElement: null  
    ...  
    hidden: false  
    id: "desc"  
    inert: false  
    innerHTML: "\n      Em 2015, a ONU propôs aos seus  
    innerText: "Em 2015, a ONU propôs aos seus países memb  
    inputMode: ""  
    isConnected: true  
    isContentEditable: false  
    lang: ""  
    ▶ lastChild: text  
    lastElementChild: null
```

Embora muitas propriedades estejam com valor null, percorra as demais propriedades e veja que muitas delas estão com valores preenchidos, como é o caso da propriedade innerHTML e textContent, que retorna o texto apresentado no parágrafo.

Você pode acessar e manipular esses elementos por meio da coleção usando índices, assim como faria em um vetor ou matriz. Por exemplo, para acessar o primeiro parágrafo, pode-se utilizar o seguinte comando:

```
const primeiroParagrafo = textos[0];  
console.log(primeiroParagrafo);
```

```
> const primeiroParagrafo = textos[0];  
  console.log(primeiroParagrafo);  
  
▼ <p id="desc">  
  " Em 2015, a ONU propôs aos seus países membros uma nova  
  Objetivos de Desenvolvimento Sustentável (ODS). "  
  </p>  
← undefined
```

Uma outra possibilidade para acessar um elemento HTML é você já definir diretamente na chamada do método getElementsByTagName qual é o nome da tag que deseja, bem como o índice e a propriedade, conforme exemplo a seguir.

```
console.log(document.getElementsByTagName("p")[1].innerHTML);  
console.log(document.getElementsByTagName("h2")[0]);
```

A primeira linha irá retornar o conteúdo da propriedade `innerHTML` do segundo parágrafo (índice 1) e a segunda linha irá retornar o texto do primeiro elemento `h2` (índice 0), conforme a figura a seguir.

```
> console.log(document.getElementsByTagName("p")[1].innerHTML);  
console.log(document.getElementsByTagName("h2")[0]);  
  
Os Objetivos de Desenvolvimento Sustentável são um a  
pobreza, proteger o meio ambiente e o clima e garant  
lugares, possam desfrutar de paz e de prosperidade.  
  
<h2>Conheça mais</h2>  
← undefined
```

`document.getElementById(id)` e `element.getAttribute()`

`document.getElementById(id)` é uma função para selecionar um elemento HTML com base no valor de seu atributo `id`. Cada elemento HTML pode ter um atributo `id` exclusivo que o identifica de maneira única na página.

Quando você invoca `document.getElementById(id)`, é retornado o elemento que possui o atributo `id` correspondente ao `id` passado como argumento.

PRATICANDO:

- Além da função `document.getElementById(id)` mencionada, há também a função `element.getAttribute(attribute)`, que permite acessar o valor de qualquer atributo específico de um elemento HTML. Você passa o nome do atributo que deseja acessar como argumento, e a função retorna o valor desse atributo. Veja um exemplo a seguir:

```
const img_ods = document.getElementById("ods-img");  
const texto_alt = img_ods.getAttribute("alt");  
console.log(texto_alt);
```

Verifique no console se foi possível recuperar o texto alternativo (atributo `"alt"`) da imagem.

```
> const img_ods = document.getElementById("ods-img");
const texto_alt = img_ods.getAttribute("alt");
console.log(texto_alt);

Logo da ODS 1

← undefined
```

Além disso, você também pode acessar o atributo diretamente pelo elemento obtido, usando a função `getElementsByTagName`, conforme exemplo a seguir.

```
function mudarCor() {
  const elemento = document.getElementsByTagName("p")[1];
  if (elemento.style.color === "red") {
    elemento.style.color = ""; // volta ao original
  } else {
    elemento.style.color = "red"; // muda para vermelho
  }
}
```

Essa ação irá gerar a alteração da cor do segundo parágrafo ao clicar no botão “Alterar cor”, conforme a figura a seguir. Observação: lembre-se também que a chamada da função ocorre pelo fato que há a chamada no evento onclick do botão (“<button onclick="mudarCor();">Alterar cor</button>”)

Para saber mais!

Assim como temos a função `document.getElementById`, para selecionar um elemento HTML com base no valor de seu atributo id, também há a função `getElementsByClassName`, que seleciona elementos HTML com base no valor de seu atributo class.

Confira mais informações sobre essa função nesse link - <https://developer.mozilla.org/pt-BR/docs/Web/API/Document/getElementsByClassName>

`document.createElement(name)`, `parentNode.appendChild(node)` e `document.createTextNode(data)`

`document.createElement(name)` é uma função para **criar um novo elemento** HTML com base no nome da tag especificado como argumento. Ela retorna um novo

elemento vazio com a tag especificada, que pode ser posteriormente personalizado e manipulado antes de ser adicionado à página.

Exemplo da criação de uma nova div, a partir da função `createElement`:

```
const novoDiv = document.createElement("div");
```

`parentNode.appendChild(node)` é uma função que permite adicionar um nó (elemento, texto, etc.) como um filho de outro nó. O nó de destino é referenciado como `parentNode`, e o nó que você deseja adicionar é referenciado como `node`. Isso é normalmente usado para adicionar o elemento criado com `createElement` (ou qualquer outro nó) a um elemento existente na página.

Veja um exemplo de como adicionar o novo `<div>` criado anteriormente a um elemento pai (por exemplo, o `<body>`):

```
var body = document.body;  
body.appendChild(novoDiv);
```

Assim, ao combinar essas duas funções, você pode criar um novo elemento com `createElement` e, em seguida, adicioná-lo como um filho a um elemento existente com `appendChild`.

`document.createTextNode(data)` é uma função para criar e inserir nós de texto em elementos HTML. Um nó de texto é uma sequência de texto que pode ser inserida em um elemento HTML para exibir texto na página.

Veja um exemplo utilizando as 3 funções mencionadas:

```
//cria um novo elemento p  
const novoParagrafo = document.createElement("p");  
//cria um texto para o elemento p  
const textoNovo = document.createTextNode("Inserindo novo conteúdo sobre ODS!");  
//faz um append do texto ao elemento p  
novoParagrafo.appendChild(textoNovo);  
//busca uma section a partir do ID "inicio"  
const conteudo = document.getElementById("inicio");  
//faz um append do parágrafo à section buscada  
conteudo.appendChild(novoParagrafo);
```

element.innerHTML

element.innerHTML é uma propriedade da API DOM para acessar ou modificar o conteúdo HTML interno de um elemento HTML específico. Essa propriedade é usada para ler ou definir o conteúdo HTML de um elemento, permitindo a manipulação do conteúdo de forma dinâmica.

Para acessar o conteúdo HTML de um elemento usando element.innerHTML, você pode simplesmente acessar a propriedade, conforme exemplo a seguir:

```
const meuElemento = document.getElementById("desc");
const conteudoHTML = meuElemento.innerHTML;
console.log(conteudoHTML);
```

```
> const meuElemento = document.getElementById("desc");
const conteudoHTML = meuElemento.innerHTML;
console.log(conteudoHTML);
```

```
Em 2015, a ONU propôs aos seus países membros
sustentável para os próximos 15 anos, a Agenda
Desenvolvimento Sustentável (ODS).
```

```
< undefined
```

No exemplo acima, conteudoHTML terá o conteúdo HTML interno do elemento parágrafo que possui o id "desc".

Você também pode usar element.innerHTML para definir o conteúdo HTML de um elemento. Isso permite que você atualize dinamicamente o conteúdo do elemento. Veja um exemplo a seguir:

```
const meuElemento = document.getElementById("desc");
meuElemento.innerHTML = "Alterando o conteúdo";
```

```
//cria um novo elemento p
const novoParagrafo = document.createElement("p");
//cria um texto para o elemento p
const textoNovo = document.createTextNode("Inserindo novo conteúdo sobre ODS!");
//faz um append do texto ao elemento p
novoParagrafo.appendChild(textoNovo);
//busca uma section a partir do ID "inicio"
const conteudo = document.getElementById("inicio");
//faz um append do parágrafo à section buscada
conteudo.appendChild(novoParagrafo);

const meuElemento = document.getElementById("desc");
meuElemento.innerHTML = "Alterando o conteúdo";
```

Inserindo novo conteúdo sobre ODS!

Objetivos de Desenvolvimento

Alterando o conteúdo

Conheça mais

Ao usar `element.innerHTML` para definir o conteúdo, ele substituirá completamente o conteúdo existente do elemento.

`element.addEventListener()`

`element.addEventListener()` é um método da API DOM para vincular funções de retorno de chamada (*event handlers*) a elementos HTML para lidar com eventos específicos. Eventos podem ser compreendidos como ações que ocorrem no navegador, como cliques de mouse, pressionamentos de tecla, carregamento de páginas, etc.

Confira a seguir uma lista com alguns exemplos de eventos que podem ser detectados:

- **blur**: quando um elemento perde o foco.
- **copy**: quando o conteúdo é copiado para a área de transferência.
- **cut**: quando o conteúdo é cortado para a área de transferência.
- **paste**: quando o conteúdo é colado da área de transferência.
- **click**: quando um elemento é clicado.
- **dblclick**: quando um elemento é clicado duas vezes rapidamente.
- **input**: quando o valor de um elemento é alterado em tempo real.
- **mouseover**: quando o ponteiro do mouse é movido sobre um elemento.
- **mouseout**: quando o ponteiro do mouse deixa a área de um elemento.
- **mousemove**: quando o ponteiro do mouse é movido dentro de um elemento.
- **mousedown**: quando um botão do mouse é pressionado sobre um elemento.
- **mouseup**: quando um botão do mouse é liberado após ter sido pressionado.
- **keydown**: quando uma tecla do teclado é pressionada.

- **keyup**: quando uma tecla do teclado é liberada após ter sido pressionada.
- **keypress**: quando uma tecla do teclado é pressionada e liberada.
- **focus**: quando um elemento recebe o foco.
- **load**: quando um recurso, como uma imagem ou uma página, é completamente carregado.
- **resize**: quando a janela do navegador é redimensionada.
- **scroll**: quando o conteúdo de um elemento que possui uma barra de rolagem é rolado.
- **submit**: quando um formulário é enviado.

O método `addEventListener()` é amplamente usado para criar interatividade em páginas da web, pois permite responder a ações do usuário e realizar ações com base nesses eventos.

PRATICANDO:

- Crie os arquivos a seguir, onde há os códigos de exemplo de uso do `addEventListener()`. Após isso, verifique o resultado via Live Server.

```
<!-- Exemplo02_aula02.html-->
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Exemplo 1</title>
  </head>
  <body>
    <h1>HTML DOM Events</h1>
    <p>Informe uma frase:</p>
    <input type="text" id="entrada_dados" />
    <p id="demo"></p>
    <script>
      const entrada = document.getElementById("entrada_dados");
      const demo = document.getElementById("demo");
      entrada.addEventListener("input", () => {
        let texto = entrada.value;
        demo.textContent = "Texto digitado: " + texto;
      });
    </script>
  </body>
</html>
```

```
<!-- Exemplo03_aula02.html-->
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Exemplo 2</title>
  </head>
  <body>
    <h1>HTML DOM Events</h1>
    <p>Mova o mouse para ver a posição:</p>
    <p id="pos_mouse"></p>
    <script>
      const posicaoMouse = document.getElementById("pos_mouse");
      document.addEventListener("mousemove", (event) => {
        const mouseX = event.clientX;
        const mouseY = event.clientY;
        posicaoMouse.textContent = `Posição X,Y do mouse: (${mouseX}, ${mouseY})`;
      });
    </script>
  </body>
</html>
```

```
<!-- Exemplo04_aula02.html-->
<!doctype html>
<html lang="pt-br">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Exemplo 3</title>
  </head>
  <body>
    <h1>HTML DOM Events</h1>
    <button id="clickBtn">Clique para Plantar uma Árvore</button>
    <p id="contagem">Árvores plantadas: <span id="valorContador">0</span></p>
    <script>
      let contador = 0;
      const clickBtn = document.getElementById("clickBtn");
      const valorContador = document.getElementById("valorContador");
      clickBtn.addEventListener("click", () => {
        contador++;
        valorContador.textContent = contador;
        alert("Você plantou uma árvore!");
      });
    </script>
  </body>
</html>
```

```
</script>
</body>
</html>
```

Explicando a função `clickBtn.addEventListener` ...

```
clickBtn.addEventListener("click", () => {
  contador++;
  valorContador.textContent = contador;
  alert("Você plantou uma árvore!");
});
```

Esse trecho usa JavaScript para reagir a um clique em um botão.

O código faz 3 coisas quando o botão é clicado:

1. aumenta o valor da variável `contador`
2. atualiza o texto mostrado na tela
3. exibe um alerta

O que é `clickBtn`

Normalmente é uma variável que guarda um botão HTML.

Em JavaScript:

```
const clickBtn = document.getElementById("btn");
```

Em HTML:

```
<button id="btn">Plantar árvore</button>
```

O que é `addEventListener`

`addEventListener` significa:

“escute um evento e execute algo quando ele acontecer”

Sintaxe geral:

```
elemento.addEventListener("evento", função)
```

O evento `"click"`

Indica que o código deve rodar quando o usuário clicar no botão.

Outros exemplos de eventos:

- "mouseover" → passar o mouse
- "keydown" → apertar tecla
- "submit" → enviar formulário

.....
O que é () => {}

Isso é uma **arrow function** (função flecha).

Ela representa a função que será executada após o clique.

Equivalente tradicional:

```
function() {  
    ...  
}
```

.....
Então () => {}

Significa: "quando clicar, execute o código abaixo"

O símbolo => é chamado de:

arrow function (função flecha)

Ele serve para criar funções de maneira mais curta.

```
clickBtn.addEventListener("click", () => {  
    contador++;  
    valorContador.textContent = contador;  
    alert("Você plantou uma árvore!");  
});
```

Utiliza uma função =>
anônima

é a mesma coisa que:

```
clickBtn.addEventListener("click", function() {  
    contador++;  
    valorContador.textContent = contador;  
    alert("Você plantou uma árvore!");  
});
```

Não utiliza a função
=>, mas continua com
uma anônima

que é a mesma coisa que:

```
function plantarArvore() {  
    contador++;  
    valorContador.textContent = contador;  
    alert("Você plantou uma árvore!");  
}  
  
clickBtn.addEventListener("click", plantarArvore);
```

Utiliza uma função
nomeada, separada da
chamada do
EventListener

Atenção:

```
clickBtn.addEventListener("click", plantarArvore);
```

Isso significa:

“quando acontecer o clique, execute a função plantarArvore”

Por que sem parênteses?

Porque você está passando a função, não executando ela imediatamente.

ERRADO

```
clickBtn.addEventListener("click", plantarArvore());
```

Isso executa a função na hora.

CERTO

```
clickBtn.addEventListener("click", plantarArvore);
```

Aqui o navegador guarda a referência da função para usar depois.

Código	Tipo
<code>function() {}</code>	função anônima
<code>function nome() {}</code>	função nomeada
<code>() => {}</code>	arrow function
<code>const x = function(){} </code>	função anônima guardada em variável

Contador

É uma variável numérica.

```
let contador = 0;
```

O operador ++ incrementa o valor em 1.

```
contador++;
```

.....
Linha 3 - valorContador.textContent = contador;

Essa linha atualiza o texto exibido na página.

Em JavaScript:

```
const valorContador = document.getElementById("contador");
```

Em HTML:

```
<span id="contador">0</span>
```

.....
O que é textContent

textContent altera o texto dentro do elemento.

Aplicando ao código:

```
valorContador.textContent = contador;
```

Se contador valer 5, o HTML vira:

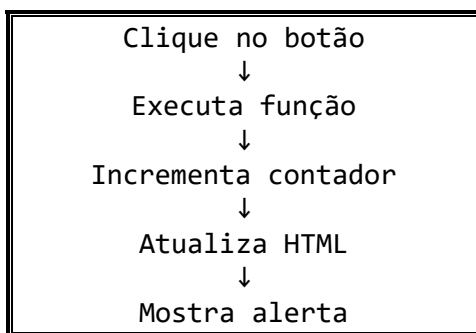
```
<span>5</span>
```

O número mostrado na tela é atualizado

.....
Linha 4 - alert("Você plantou uma árvore!");

alert() mostra uma janela pop-up do navegador.

.....
Fluxo de execução da função:



Parte	Função
addEventListener	escuta eventos
"click"	evento de clique
() => {}	função executada
contador++	soma 1
textContent	altera texto HTML
alert()	mostra pop-up